

Table des matières

1. Pour commencer
2. Premiers pas avec les sémaphores
3. Kill game

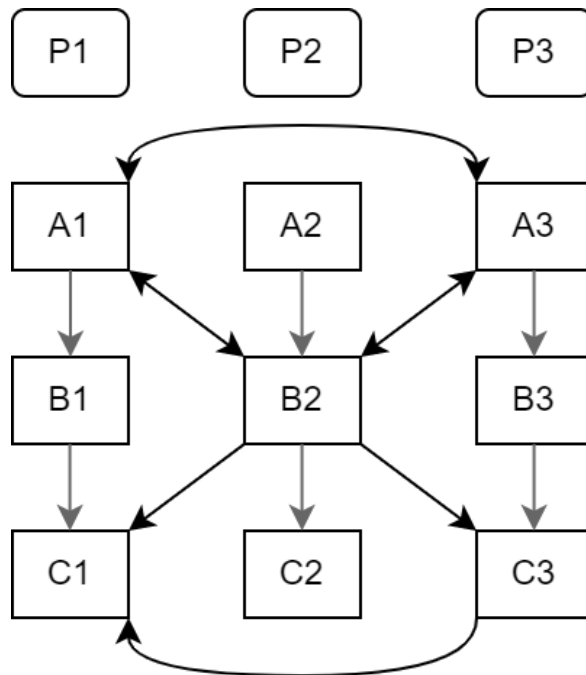
1. Pour commencer

Dans ce TP, nous allons manipuler les sémaphores IPC System V.

Comme d'habitude, n'oubliez pas de réaliser la gestion d'erreurs et utilisez un makefile...
Même si je ne suis pas sûr que beaucoup d'étudiants lisent ces consignes...

2. Premiers pas avec les sémaphores

Dans cet exercice, nous souhaitons utiliser les sémaphores pour résoudre le diagramme de précedence suivant :



1. Proposez une solution avec les sémaphores pour résoudre ce diagramme.

Nous considérons le programme ([question 1.zip](#)) qui simule l'exécution des trois processus P1, P2 et P3 par trois fils. Il vérifie si les blocs de code s'exécutent correctement suivant le diagramme ci-dessus. Si vous compilez le programme et que vous l'exécutez, des erreurs sont sans doute affichées. Ci-dessous, on obtient un exemple d'exécution. Le bloc A3 s'exécute en même temps que le bloc A1, ce qui provoque une erreur (ils sont normalement en exclusion mutuelle).

```
Start block #A1 [OK]
Start block #A2 [OK]
Start block #A3 [ERROR]
End block #A2
End block #A3
End block #A1
Start block #B2 [OK]
Start block #B3 [OK]
End block #B2
Start block #B1 [OK]
End block #B3
End block #B1
Start block #C2 [OK]
Start block #C3 [OK]
End block #C3
End block #C2
Start block #C1 [OK]
End block #C1
Execution failed (it sucks!).
```

2. Modifiez le programme en ajoutant la création du tableau de sémaphores (dans le main) et les opérations sur les sémaphores (P et V) aux bons endroits dans les fonctions `childx` avant ou après les blocs concernés, à la place de `#TODO#`.

Tous les `#TODO#` ne correspondent pas forcément à des opérations !

3. Kill game

Nous souhaitons écrire une application basée sur deux programmes : un *coordinateur* et un *client*.

Le *client* doit être exécuté au maximum X fois. Une fois l'ensemble des *clients* démarrés et que chacun possède un numéro unique, le jeu démarre. Chaque *client* attend pendant un temps aléatoire. Puis il choisit un adversaire encore en vie et le tue, puis attend à nouveau. Après sa pause, s'il s'aperçoit qu'il est mort ou qu'il est le dernier en vie, il s'arrête.

Le *coordinateur* crée un tableau de sémaphores et l'initialise. Il attend simplement que l'ensemble des clients soient morts.

Le nombre maximum de clients (X) et la clé du tableau de sémaphores sont fixés par des constantes.

Vous ne devez utiliser aucun moyen de communication entre les processus autre que les sémaphores.

1. Comment faire en sorte qu'il n'y ait pas plus de X *clients* simultanément ? Comment faire en sorte qu'un *client* ne soit pas bloqué au démarrage si le nombre maximum de clients a été atteint ?
2. Comment un *client* peut-il trouver un numéro unique ?
3. Comment tuer un *client* ?
4. Écrivez les deux programmes.

Rappel : la fonction `semctl` permet de récupérer la valeur d'un sémaphore. Mais contrairement aux opérations (avec `semop`), cette solution n'est pas sûre... à moins d'utiliser une section critique !